

GPTIMER

GPTIMER

- 1. GPTimer Overview
- 2. Core Concepts
 - 2.1 Clock Source and Resolution
 - 2.2 Counting Direction
 - 2.3 Pure Counting Mode vs Alarm Mode
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
- 5. Source Code Analysis
 - 5.1 Macro Definitions and Global Variables
 - 5.2 timer_init() — Three-Step Start
 - 5.3 app_main() — Main Entry
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Output
- 7. Common Issues

1. GPTimer Overview

GPTimer (General Purpose Timer) is a hardware timer peripheral of ESP32-S3, providing an independent 54-bit counter with configurable clock source, resolution, counting direction, and alarm threshold. Compared to software timer (ESP_Timer), GPTimer directly operates hardware registers, providing **maximum flexibility**.

This experiment configures GPTimer as a 1MHz free-running up counter (no interrupt, no callback). Main loop reads the current count value once per second. Demonstrates GPTimer's most basic "pure counting" usage—a high-precision time reference.

ESP_Timer vs GPTimer Selection Guide:

Feature	esp_timer (software timer)	GPTimer (hardware timer)
Abstraction Level	High (ESP-IDF encapsulation)	Low (direct hardware operation)
Usage Complexity	Simple (3-5 APIs)	Medium (7+ APIs)
Callback Support	Built-in ISR/TASK dispatch	Need to configure ISR yourself
Resolution	1μs	Configurable (up to 40MHz)
Power Consumption	Fixed	Configurable clock source to reduce power
Applicable Scenario	General timing needs	High-precision timing, encoder interface, PWM generation

Typical Application Scenarios:

Application Type	Example
Precise Timing	Motor PWM deadzone control, stepper motor pulse sequences
Frequency Measurement	Input capture frequency measurement, tachometer signal processing
Pulse Generation	Precise delay trigger, laser/ultrasonic ranging trigger signal
High-speed Counting	Encoder pulse counting, production line photoelectric counting

2. Core Concepts

2.1 Clock Source and Resolution

This configuration struct is like "factory settings" for a stopwatch—choosing which clock source, which direction to count, and what each tick represents. 1MHz resolution means 1 tick equals 1 microsecond. The read count value is directly microseconds, very intuitive.

```
gptimer_config_t timer_config = {
    // Default clock source (XTAL-derived)
    .clk_src      = GPTIMER_CLK_SRC_DEFAULT,
    .direction    = GPTIMER_COUNT_UP,        // Count up
    // Resolution 1MHz, 1 tick = 1µs
    .resolution_hz = 1000000,
};
```

`resolution_hz` determines what time each tick represents:

resolution_hz	1 tick =	54-bit max count time
1,000,000 (1MHz)	1 µs	~570,000 years
1,000 (1kHz)	1 ms	~570 million years
40,000,000 (40MHz)	25 ns	~14,000 years

Clock source precision affects counting precision:

Clock Source Option	Precision	Description
<code>GPTIMER_CLK_SRC_DEFAULT</code>	~±20ppm	XTAL crystal clock, most stable
<code>GPTIMER_CLK_SRC_RC_FAST</code>	~±5%	Internal RC oscillator, lower precision

2.2 Counting Direction

This enum tells the timer "which direction to count"—either from large to small (countdown) or from small to large (count-up). This experiment chooses count-up, so the read value directly equals microseconds elapsed since start, no subtraction needed.

```
typedef enum {
    // Count down: from alarm value decrementing to 0
    GPTIMER_COUNT_DOWN = 0,
    // Count up: from 0 incrementing to alarm value
    GPTIMER_COUNT_UP    = 1,
} gptimer_count_direction_t;
```

This experiment chooses `GPTIMER_COUNT_UP`: starting from 0 and incrementing, no alarm value limit, continuously free counting. The read count value directly represents "microseconds elapsed since startup".

2.3 Pure Counting Mode vs Alarm Mode

Mode	Configuration	Read Method	This Experiment
Pure counting	No alarm, no callback	Software polling <code>gptimer_get_raw_count()</code>	
Alarm counting	Set alarm value + ISR callback	ISR notification + e.g. PWM generation	—

Pure counting mode advantages: simple, zero interrupt overhead, CPU reads when needed, suitable for timestamp and time difference measurement.

3. Project Architecture and Flow

```
app_main()
├─ timer_init()           Create 1MHz free counter
│   ├─ gptimer_new_timer() Create timer instance (clock source+direction+resolution)
│   ├─ gptimer_enable()    Enable timer (power on but not counting)
│   └─ gptimer_start()     Start counting (from 0, incrementing)
└─ while(1) Main Loop
    ├─ gptimer_get_raw_count() Read current count value (64-bit)
    ├─ ESP_LOGI()             Display in three units: count / μs / ms
    └─ vTaskDelay(1000ms)     Read once per second
```

GPTimer Lifecycle:

```

gptimer_new_timer()      Create + allocate resources
└─ gptimer_enable()      Enable clock + power on
    └─ gptimer_start()    Start counting (direction+reset to 0)
        |
        └─ ... in use (this experiment never exits)...
            |
            └─ gptimer_stop() Stop counting
                └─ gptimer_disable() Disable clock
                    └─ gptimer_del_timer() Release resources

```

4. Hardware Dependencies

Hardware Connection: No external hardware dependency, pure timer register operation.

Dependencies: freertos, driver/gptimer, esp_log

5. Source Code Analysis

5.1 Macro Definitions and Global Variables

These two lines are preparation—define a log tag for easy identification during debugging, then define an empty handle variable to store the timer instance later. The handle is like a "remote control", all future timer operations rely on it to specify which timer.

```

#define TAG "TIMER_COUNT" // Log tag
// Timer handle
static gptimer_handle_t gptimer = NULL;

```

Handle is ESP-IDF's universal design pattern: creation API returns an **opaque pointer**, all subsequent operations reference the peripheral instance through it. The handle internally points to driver data structure allocated by ESP-IDF, users don't need to care about its internal layout.

5.2 timer_init() — Three-Step Start

This function starts the timer with "three-step process": first fill the configuration table, call new to create timer instance (like buying a stopwatch); second enable to power on the timer (install battery); third start to begin counting (press start button). Order cannot be changed, all three are essential.

```

static void timer_init(void)
{
    gptimer_config_t timer_config = {
        // Default clock source (XTAL-derived)
        .clk_src      = GPTIMER_CLK_SRC_DEFAULT,
        .direction     = GPTIMER_COUNT_UP,        // Count up
        // Resolution 1MHz, 1 tick = 1µs
        .resolution_hz = 1000000,
    };
    // ① Create timer instance
    gptimer_new_timer(&timer_config, &gptimer);
}

```

```

// ② Enable timer
gptimer_enable(gptimer);
// ③ Start counting
gptimer_start(gptimer);
}

```

Why choose 1MHz instead of 40MHz? 1 tick = 1 μ s is a very intuitive relationship—count value is directly microseconds, no conversion needed. 40MHz (25ns/tick) has excessive precision but troublesome conversion, and this experiment's `vTaskDelay(1000ms)` polling-level precision doesn't need such high resolution.

Why not split into two steps (enable → delay → start)? Some peripherals need stabilization time after clock enable. GPTimer internal hardware is synchronous design, register writes from `gptimer_enable` to `gptimer_start` are guaranteed ordering by APB bus bridge, no additional delay needed.

5.3 app_main() — Main Entry

Main function logic is very simple: first print a tagline, then call `timer_init` to start the timer, then enter an infinite loop—every 1 second read the timer's current count value, then print in three units (raw count/ μ s/ms). The three numbers are essentially the same, just different units, making it convenient to understand the correspondence between tick and time.

```

void app_main(void)
{
    // Print startup info
    ESP_LOGI(TAG, "ESP32-S3 GPTimer Raw Counting Mode");
    // Initialize timer
    timer_init();
    // Confirm started
    ESP_LOGI(TAG, "Timer started successfully, counting freely");

    while (1) {
        uint64_t count_value;
        // Read current count value
        gptimer_get_raw_count(gptimer, &count_value);

        // Display in 3 units: raw/ $\mu$ s/ms
        ESP_LOGI(TAG, "Count=%llu | %llu us | %.2f ms",
                 count_value, count_value, (float)count_value / 1000.0f);

        // Delay 1 second
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

```

Output Interpretation:

Count=1006516		1006516 us		1006.52 ms
↑		↑		↑
Raw count value		Same value= μ s		count÷1000 = ms

The three columns in output are essentially the same value: Since 1 tick = 1µs, `count_value` itself is microseconds. The purpose of three-column display is to help understand the correspondence between "1 tick = 1µs". If configured as 40MHz, need to divide by 40 to convert time.

First output offset explanation: First line count ≈ 1006516 instead of 1000000 is because from chip power-on to first read, there's startup time for bootloader → FreeRTOS initialization → `app_main()` → `timer_init()`. The timer was already counting during this time. Each subsequent increment is precisely 1,000,000 because GPTimer uses XTAL crystal clock source with precision ±20ppm, far better than `vTaskDelay`'s software delay precision.

6. Operation Flow

6.1 Build and Flash

For detailed flashing process, see: 1. Before Development → 3. Build and Flash

6.2 Normal Output

```
I (275) TIMER_COUNT: ESP32-S3 GPTimer Raw Counting Mode
I (275) TIMER_COUNT: Timer started successfully, counting freely
I (275) TIMER_COUNT: Count=2583 | 2583 us | 2.58 ms
I (1285) TIMER_COUNT: Count=1006516 | 1006516 us | 1006.52 ms
I (2285) TIMER_COUNT: Count=2006516 | 2006516 us | 2006.52 ms
I (3285) TIMER_COUNT: Count=3006516 | 3006516 us | 3006.52 ms
I (4285) TIMER_COUNT: Count=4006516 | 4006516 us | 4006.52 ms
```

Observing the pattern:

```
I (275) TIMER_COUNT: ESP32-S3 GPTimer Raw Counting Mode
I (275) TIMER_COUNT: Timer started successfully, counting freely
I (275) TIMER_COUNT: Count=2583 | 2583 us | 2.58 ms
I (1285) TIMER_COUNT: Count=1006516 | 1006516 us | 1006.52 ms
I (2285) TIMER_COUNT: Count=2006516 | 2006516 us | 2006.52 ms
I (3285) TIMER_COUNT: Count=3006516 | 3006516 us | 3006.52 ms
I (4285) TIMER_COUNT: Count=4006516 | 4006516 us | 4006.52 ms
I (5285) TIMER_COUNT: Count=5006516 | 5006516 us | 5006.52 ms
I (6285) TIMER_COUNT: Count=6006516 | 6006516 us | 6006.52 ms
I (7285) TIMER_COUNT: Count=7006516 | 7006516 us | 7006.52 ms
I (8285) TIMER_COUNT: Count=8006516 | 8006516 us | 8006.52 ms
I (9285) TIMER_COUNT: Count=9006516 | 9006516 us | 9006.52 ms
I (10285) TIMER_COUNT: Count=10006516 | 10006516 us | 10006.52 ms
I (11285) TIMER_COUNT: Count=11006516 | 11006516 us | 11006.52 ms
I (12285) TIMER_COUNT: Count=12006516 | 12006516 us | 12006.52 ms
I (13285) TIMER_COUNT: Count=13006516 | 13006516 us | 13006.52 ms
I (14285) TIMER_COUNT: Count=14006516 | 14006516 us | 14006.52 ms
```

- First line count is small (2583), because from `timer_init()` to first `gptimer_get_raw_count()` only a few lines of code apart, so only ~2.5ms interval
- Each subsequent increment is precisely 1,000,000, corresponding to 1 second. GPTimer uses XTAL crystal clock with precision ±20ppm, far more reliable than `vTaskDelay`
- Timestamps in log brackets (1285 → 2285 → 3285) are also exactly 1000ms apart, indicating

`vTaskDelay(1000ms)` also has good precision under ESP-IDF default 1000Hz tick

7. Common Issues

Phenomenon	Cause	Solution
Count faster/slower than expected	<code>resolution_hz</code> misconfigured	Check <code>resolution_hz = 1000000</code>
<code>gptimer_new_timer</code> returns <code>ESP_ERR_NOT_FOUND</code>	Hardware timer resources exhausted (max 4)	Release unused timers
<code>gptimer_get_raw_count</code> reads 0	<code>gptimer_start()</code> not called	In order: new → enable → start
Count value increment inaccurate	<code>vTaskDelay</code> precision limited ($\pm 10\text{ms}$)	Use alarm+ISR mode instead of polling, or read <code>esp_timer_get_time</code> for comparison
Output timestamp and counter out of sync	Main loop printing is after <code>vTaskDelay</code> returns, not exactly 1000ms	Normal phenomenon, <code>vTaskDelay</code> guarantees "at least delay" not precise delay